

面向对象程序设计

一、选择题

1.在 Java 中，以下哪个关键字用于定义类？

A. class B. interface C. enum D. package

2.下列代码执行结果是。

```
String s = "abcd";
```

```
String s1 = new String(s);
```

```
if (s == s1)
```

```
    System.out.println("the same");
```

```
if (s.equals(s1))
```

```
    System.out.println("equals");
```

A. the same equals B. equals C. the same D. 什么结果都不输出

3.下列关于 Java 中异常处理的说法，哪个是不正确的？

A. try 块中可以匹配多个 catch 块。

B. finally 块中的代码总是会执行，除非 JVM 退出。

C. throw 语句用于抛出异常对象。

D. throws 关键字用于声明一个方法可能抛出的异常类型，并且这些异常类型必须在方法内部被捕获。

4.下列关于 Java 中 ArrayList 的说法，哪个是不正确的？

A. ArrayList 是基于数组实现的。

B. ArrayList 是线程安全的。

C. ArrayList 可以动态调整大小。

D. ArrayList 允许存储 null 元素。

5.在 Java 中，多态性主要通过什么实现？

A. 方法的重载 B. 类的继承 C. 抽象类 D. 方法的重写和接口的实现

6.在 Java 中，如果一个类没有显式地声明一个构造方法，那么会。

A. 编译错误

B. 运行时错误

- C. 提供一个默认的无参数构造方法
- D. 继承父类的构造方法

7. 在 Java 中，以下哪个集合类不允许有重复的元素，并且元素的顺序是插入顺序？

- A. HashSet B. TreeSet C. LinkedHashSet D. LinkedList

8. 以下哪个代码片段会导致编译错误？

- A. `abstract class A { abstract void m(); } class B extends A { }`
- B. `abstract class A { abstract void m(); } class B extends A { void m() { } }`
- C. `abstract class A { abstract void m(int x); } class B extends A { void m(int x) { } }`
- D. `abstract class A { abstract void m(); } abstract class B extends A { }`

9. 以下哪个关键字用于同步方法，以确保在同一时间内只有一个线程可以执行该方法？

- A. static B. synchronized C. final D. volatile

10. 以下哪个事件监听器接口用于处理窗口关闭事件？

- A. WindowListener B. ActionListener C. MouseListener D. KeyAdapter

11. 当一个方法通过 `throws` 声明抛出了一个异常，但调用该方法的代码没有捕获这个异常，会发生什么？

- A. 程序将崩溃并显示错误消息
- B. 异常会被自动捕获并处理
- C. 方法调用将失败，但不会影响程序的其他部分
- D. 程序将继续执行，忽略该异常

12. 以下哪个说法是正确的？

- A. 线程一旦启动，就会立即执行
- B. 线程的状态可以是新建、就绪、运行、阻塞、死亡
- C. 线程优先级越高，它就越早执行
- D. 线程死亡后，可以重新启动

13. 以下哪个布局管理器将组件按行或列进行排列，并允许每个组件占据相同大小的空间？

- A. BorderLayout B. GridLayout C. BoxLayout D. FlowLayout

14. 以下哪个代码片段展示了抽象类与接口之间的正确关系？（ ）

- A. `interface I { void m(){}; } abstract class A implements I { }`

- B. `abstract class A { abstract void m(); } interface I extends A { }`
- C. `interface I { } abstract class A implements I { abstract void m(); }`
- D. `interface I { void m(); } abstract class A implements I { void m() { } }`

15. 以下关于 Java 中继承的描述，错误的是（ ）。

- A. Java 中的继承允许一个子类继承多个父类
- B. 子类可作为另一个子类的父类
- C. 当实例化子类时会调用父类中的构造方法
- D. 子类可以访问父类的公共属性

二、填空题

1. 在 Java 中，使用关键字_____可以确保变量只能被赋值一次，并且之后不能被改变。
2. 在 Java 中，当一个类实现了某个接口但并未实现该接口中的所有方法时，该类必须被声明为_____类。
3. _____方法是一种特殊的方法，用于对象的初始化，并且该方法在对象被创建时自动调用。
4. 在多线程中，调用_____方法，让正在执行的线程休眠一段时间。
5. 计算字符串的长度使用_____方法。
6. 面向对象的三个特性是：封装性、继承性和_____。
7. 阅读以下程序，将程序补充完整。

```
public class Program06 {  
    public static String sort(String s) {  
        char[] ch = s._____;  
        for (int i = 0; i < ch.length - 1; i++) {  
            for (int j = i + 1; j < ch.length; j++) {  
                if (ch[j] > ch[i]) {  
                    char t = _____;  
                    ch[j] = ch[i];  
                    ch[i] = t;  
                }  
            }  
        }  
    }  
}
```

```

        }
    }
}

String str = new String(ch);

return str;
}

public static void main(String[] args) {
    String s = "morning";
    System.out.println(sort(s));
}
}

```

8. 阅读以下程序，将程序补充完整。

```

class BaseClass {
    void show() {
        System.out.println("Base class show method.");
    }
}

class DerivedClass extends BaseClass {
    @Override
    void show() {
        System.out.println("Derived class show method.");
    }

    void display() {
        super._____; // Should call BaseClass's show method
    }
}

```

```
}
```

```
public class Main {  
  
    public static void main(String[] args) {  
  
        DerivedClass obj = new DerivedClass();  
  
        obj._____; // Should print "base class show method."  
  
    }  
}
```

三、问答题

- 1.在 Java 中，子类重写父类方法时需要注意哪些规则？
- 2.什么是向上转型，向上转型在实现多态中起到了什么作用。
- 3.Java 中的接口（Interface）和抽象类（Abstract Class）有什么区别。

四、程序分析题

- 1、阅读下列代码，分析该程序的作用，思考这段代码是否有错误，如果有错误请指出该错误，并改正相关代码。

```
public class Person implements Comparable<Person> {  
  
    private String name;  
  
    private int age;  
  
    public Person(String name, int age) {  
  
        this.name = name;  
  
        this.age = age;  
  
    }  
  
    public int compareTo(Object o) {  
  
        if (o instanceof Person) {  
  
            Person p = (Person) o;
```

```

        return Integer.compare(this.age, p.age);
    }

    return -1;
}

// getters and setters 省略
}

```

2、分析以下 Java 代码，说明其输出结果，并解释原因。

```

public class Test {

    static int count = 0;

    public Test() {

        count++;
    }

    public static void main(String[] args) {

        Test[] tests = new Test[5];

        for (int i = 0; i < tests.length; i++) {

            if (i % 2 == 0) {

                tests[i] = new Test();
            }
        }

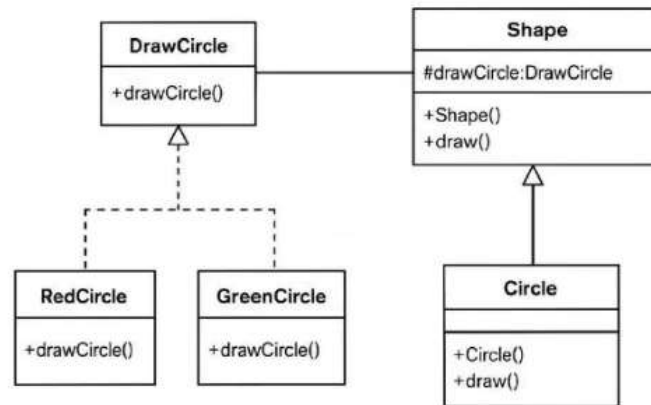
        System.out.println(count);
    }

}

```

3、分析以下代码，并补全代码。

以下 Java 代码实现一个简单绘图工具，绘制不同形状以及不同颜色的图形。部分接口、类及其关系如图所示。



代码如下:

```
interface DrawCircle {    // 绘制圆形
```

```
    public _____;
}
```

```
class RedCircle implements DrawCircle {    // 绘制红色圆形
```

```
    public void drawCircle(int radius, int x, int y) {

        System.out.println("Drawing Circle[red, radius: " + radius + ", x: " + x + ", y: " + y
+ "]" );
    }
}
```

```
class GreenCircle implements DrawCircle {    // 绘制绿色圆形
```

```
    public void drawCircle(int radius, int x, int y) {

        System.out.println("Drawing Circle[green, radius: " + radius + ", x: " + x + ", y: " +
y + "]" );
    }
}
```

```

abstract class Shape {    // 形状

    protected _____;

    public Shape(DrawCircle drawCircle) {

        this.drawCircle = drawCircle;

    }

    public abstract void draw();

}

```

```

class Circle extends Shape {    // 圆形

    private int x, y, radius;

    public Circle(int x, int y, int radius, DrawCircle drawCircle) {

        _____;

        this.x = x;

        this.y = y;

        this.radius = radius;

    }

    public void draw() {

        drawCircle._____;

    }

}

```

```

public class DrawCircleMain {

    public static void main(String[] args) {

        Shape redCircle = new Circle(100, 100, 10, _____); // 绘制红色圆形

        Shape greenCircle = new Circle(200, 200, 10, _____); // 绘制绿色圆形

    }

}

```



```
        redCircle.draw();

        greenCircle.draw();
    }
}
```

五、程序设计

编写一个 Java 程序，要求实现以下功能：

1. 定义一个 `Student` 类，包含学号（`sno`）、姓名（`sname`）和成绩（`grade`）属性，并提供相应的 `getter` 和 `setter` 方法。

设置属性为私有状态，通过公开的 `getter` 或 `setter` 方法获得或设置相应属性值。

2. 定义一个 `StudentManager` 类，实现以学生列表作为参数来计算平均成绩的方法 `avg_grade()`。

3. 在 `StudentManager` 类中，实现显示学生列表的方法 `display()`

4. 在主类中设计一个集合，添加五名 `Student` 信息，计算平均成绩，并调用 `display` 方法展示学生列表，输出到控制台上。

5. 实现方法 `writeToFile`，将上述计算所得平均成绩和展示的学生列表写入指定的文本文件中，文件相对路径是 `\data\output.txt`。

最终展示结果如图所示。

```
平均成绩: 86.8
学号: 001 姓名: 张一 成绩: 85.5
学号: 002 姓名: 李四 成绩: 92.0
学号: 003 姓名: 王五 成绩: 78.0
学号: 004 姓名: 赵六 成绩: 88.5
学号: 005 姓名: 孙七 成绩: 90.0
```